Question 1. What Is A Compiler?

Answer : A compiler is a program that reads a program written in one language –the source language and translates it into an equivalent program in another language-the target language. The compiler reports to its user the presence of errors in the source program.

Question 2. What Are The Two Parts Of A Compilation? Explain Briefly.

Answer : Analysis and Synthesis are the two parts of compilation.

- The analysis part breaks up the source program into constituent pieces and creates an intermediate representation of the source program.
- The synthesis part constructs the desired target program from the intermediate representation.

Question 3. List The Sub Parts Or Phases Of Analysis Part ?

Answer : Analysis consists of three phases:

- Linear Analysis.
- Hierarchical Analysis.
- Semantic Analysis.

Question 4. Suggest A Suitable Approach For Computing Hash Function?

Answer : Using hash function we should obtain exact locations of name in symbol table.

- The hash function should result in uniform distribution of names in symbol table.
- The hash function should be such that there will be minimum number of collisions.
- Collision is such a situation where hash function results in same location for storing the names.

Question 5. What Is Code Motion?

Answer : Code motion is an optimization technique in which amount of code in a loop is decreased. This transformation is applicable to the expression that yields the same result independent of the number of times the loop is executed. Such an expression is placed before the loop.

Question 6. What Are The Properties Of Optimizing Compiler?

Answer : The source code should be such that it should produce minimum amount of target code.

There should not be any unreachable code.

Dead code should be completely removed from source language.

The optimizing compilers should apply following code improving transformations on source language.

- common sub expression elimination
- dead code elimination
- code movement
- strength reduction

Question 7. What Are The Various Ways To Pass A Parameter In A Function?

Answer :

- Call by value
- Call by reference
- Copy-restore
- Call by name

Question 8. Mention The Issues To Be Considered While Applying The Techniques For Code Optimization.

Answer :

- The semantic equivalence of the source program must not be changed.
- The improvement over the program efficiency must be achieved without changing the algorithm of the program.
- The machine dependent optimization is based on the characteristics of the target machine for the instruction set used and addressing modes used for the instructions to produce the efficient target code.
- The machine independent optimization is based on the characteristics of the programming languages for appropriate programming structure and usage of efficient arithmetic properties in order to reduce the execution time.
- Available expressions.
- Reaching definitions.
- Live variables.
- Busy variables.

Question 9. What Are The Basic Goals Of Code Movement?

Answer :

•To reduce the size of the code i.e. to obtain the space complexity.

•To reduce the frequency of execution of code i.e. to obtain the time complexity.

Question 10. List The Different Storage Allocation Strategies?

Answer :

The strategies are:

- Static allocation
- Stack allocation
- Heap allocation

Question 11. What Are The Contents Of Activation Record?

Answer : The activation record is a block of memory used for managing the information needed by a single execution of a procedure.

Various fields f activation record are:

- Temporary variables
- Local variables
- Saved machine registers
- Control link
- Access link
- Actual parameters
- Return values

Question 12. What Is Dynamic Scoping?

Answer :

In dynamic scoping a use of non-local variable refers to the non-local data declared in most recently called and still active procedure. Therefore each time new findings are set up for local names called procedure. In dynamic scoping symbol tables can be required at run time.

Question 13. Define Symbol Table.

Answer :

Symbol table is a data structure used by the compiler to keep track of semantics of the variables. It stores information about scope and binding information about names.

1. **Question 14. What Is Linear Analysis?**

   Answer :

   Linear analysis is one in which the stream of characters making up the source program is read from left to right and grouped into tokens that are sequences of characters having a collective meaning.Also called lexical analysis or scanning.

2. **Question 15. List The Various Phases Of A Compiler ?**

   Answer :

   The following are the various phases of a compiler:

   - Lexical Analyzer
   - Syntax Analyzer
   - Semantic Analyzer
   - Intermediate code generator
   - Code optimizer
   - Code generator

1. **Question 16. What Are The Classifications Of A Compiler?**

   Answer :

   Compilers are classified as:

   - Single- pass
   - Multi-pass
   - Load-and-go
   - Debugging or optimizing

2. **Question 17. What Is A Symbol Table?**

   Answer :

   A symbol table is a data structure containing a record for each identifier, with fields for the attributes of the identifier. The data structure allows us to find the record for each identifier quickly and to store or retrieve data from that record quickly. Whenever an identifier is detected by a lexical analyzer, it is entered into the symbol table. The attributes of an identifier cannot be determined by the lexical analyzer.

Question 18. Mention Some Of The Cousins Of A Compiler.

Answer :

Cousins of the compiler are:

- Preprocessors
- Assemblers
- Loaders and Link-Editors

1. **Question 19. List The Phases That Constitute The Front End Of A Compiler.**

   Answer :

   The front end consists of those phases or parts of phases that depend primarily on the source language and are largely independent of the target machine. These include

   · **Lexical and Syntactic analysis**

   · **The creation of symbol table**

   · **Semantic analysis**

   · **Generation of intermediate code**

   A certain amount of code optimization can be done by the front end as well. Also includes error handling that goes along with each of these phases.

2. **Question 20. Mention The Back-end Phases Of A Compiler.**

   Answer :

   The back end of compiler includes those portions that depend on the target machine and generally those portions do not depend on the source language, just the intermediate language. These include

   · Code optimization

   · Code generation, along with error handling and symbol- table operations.

Question 21. Define Compiler-compiler.

Answer :

Systems to help with the compiler-writing process are often been referred to as compiler-compilers, compiler-generators or translator-writing systems.
Largely they are oriented around a particular model of languages , and they are suitable for generating compilers of languages similar model.

Question 22. List The Various Compiler Construction Tools.

Answer :

The following is a list of some compiler construction tools:

- Parser generators
- Scanner generators
- Syntax-directed translation engines
- Automatic code generators
- Data-flow engines

Question 23. Differentiate Tokens, Patterns, Lexeme.

Answer :

Tokens- Sequence of characters that have a collective meaning.
Patterns- There is a set of strings in the input for which the same token is produced as output. This set of strings is described by a rule called a pattern associated with the token
Lexeme- A sequence of characters in the source program that is matched by the pattern for a token.

Question 24. List The Operations On Languages.
Answer :

- **Union** – L U M = {s | s is in L or s is in M}
- **Concatenation** – LM = {st | s is in L and t is in M}
- **Kleene Closure** – L^* (zero or more concatenations of L)
- **Positive Closure** – L+ (one or more concatenations of L)

Question 25. Write A Regular Expression For An Identifier.
Answer :
An identifier is defined as a letter followed by zero or more letters or digits.
The regular expression for an identifier is given as
letter (letter | digit)*
Question 26. Mention The Various Notational Short Hands For Representing Regular Expressions.
Answer :

- One or more instances (+)
- Zero or one instance (?)
- Character classes ([abc] where a,b,c are alphabet symbols denotes the regular expressions a | b | c.)
- Non regular sets

Question 27. What Is The Function Of A Hierarchical Analysis?
Answer :
Hierarchical analysis is one in which the tokens are grouped hierarchically into nested collections with collective meaning.Also termed as Parsing.

Question 28. What Does A Semantic Analysis Do?
Answer :
Semantic analysis is one in which certain checks are performed to ensure that components of a program fit together meaningfully.Mainly performs type checking.

Question 29. List The Various Error Recovery Strategies For A Lexical Analysis.
Answer :
Possible error recovery actions are:

- Panic mode recovery
- Deleting an extraneous character
- Inserting a missing character

- Replacing an incorrect character by a correct character
- Transposing two adjacent characters

Question 30. Mention The Basic Issues In Parsing.

Answer :

There are two important issues in parsing.

- Specification of syntax
- Representation of input after parsing.

Question 31. Why Lexical And Syntax Analysers Are Separated Out?

Answer :

Reasons for separating the analysis phase into lexical and syntax analyzers:

- Simpler design.
- Compiler efficiency is improved.
- Compiler portability is enhanced.

Question 32. Define A Context Free Grammar.

Answer :

A context free grammar G is a collection of the following
· V is a set of non terminals
· T is a set of terminals
· S is a start symbol
· P is a set of production rules
G can be represented as G = (V,T,S,P)
Production rules are given in the following form
Non terminal → (V U T)*

Question 33. Briefly Explain The Concept Of Derivation.

Answer :

Derivation from S means generation of string w from S. For constructing derivation two things are important.
i) Choice of non terminal from several others.
ii) Choice of rule from production rules for corresponding non terminal.
Instead of choosing the arbitrary non terminal one can choose
i) either leftmost derivation – leftmost non terminal in a sentinel form.
ii) or rightmost derivation – rightmost non terminal in a sentinel form.

Question 34. Define Ambiguous Grammar.

Answer :

A grammar G is said to be ambiguous if it generates more than one parse tree for some sentence of language L(G).
i.e. both leftmost and rightmost derivations are same for the given sentence.

Question 35. What Is A Operator Precedence Parser?

Answer :

A grammar is said to be operator precedence if it possess the following properties:
1. No production on the right side is ε.
2. There should not be any production rule possessing two adjacent non terminals at the right hand side.

Question 36. List The Properties Of Lr Parser ?

Answer :

1. LR parsers can be constructed to recognize most of the programming languages for which the context free grammar can be written.
2. The class of grammar that can be parsed by LR parser is a superset of class of grammars that can be parsed using predictive parsers.
3. LR parsers work using non backtracking shift reduce technique yet it is efficient one.

Question 37. Mention The Types Of Lr Parser ?

Answer :

- SLR parser- simple LR parser
- LALR parser- lookahead LR parser
- Canonical LR parser

Question 38. What Are The Problems With Top Down Parsing?

Answer :

The following are the problems associated with top down parsing:

- Backtracking
- Left recursion
- Left factoring
- Ambiguity

Question 39. Write Short Notes On Yacc ?

Answer :

- YACC is an automatic tool for generating the parser program.
- YACC stands for Yet Another Compiler Compiler which is basically the utility available from UNIX.
- Basically YACC is LALR parser generator.

Question 40. What Is Meant By Viable Prefixes?

Answer :

The set of prefixes of right sentential forms that can appear on the stack of a shift-reduce parser are called viable prefixes. An equivalent definition of a viable prefix is that it is a prefix of a right sentential form that does not continue past the right end of the rightmost handle of that sentential form.

Question 41. Define Handle ?

Answer :

A handle of a string is a substring that matches the right side of a production, and whose reduction to the nonterminal on the left side of the production represents one step along the reverse of a rightmost derivation.

A handle of a right – sentential form γ is a production A→β and a position of γ where the string β may be found and replaced by A to produce the previous right-sentential form in a rightmost derivation of γ. That is , if S =>αAw =>αβw,then A→β in the position following α is a handle of αβw.

Question 42. What Are Kernel & Non-kernel Items?

Answer :

Kernel items, whish include the initial item, S' → .S, and all items whose dots are not at the left end.

Non-kernel items, which have their dots at the left end.

Question 43. What Is Phrase Level Error Recovery?

Answer :

Phrase level error recovery is implemented by filling in the blank entries in the predictive parsing table with pointers to error routines. These routines may change, insert, or delete symbols on the input and issue appropriate error messages. They may also pop from the stack.

Question 44. What Are The Benefits Of Intermediate Code Generation?

Answer :

- A Compiler for different machines can be created by attaching different back end to the existing front ends of each machine.
- A Compiler for different source languages can be created by proving different front ends for corresponding source languages t existing back end.
- A machine independent code optimizer can be applied to intermediate code in order to optimize the code generation.

Question 45. What Are The Various Types Of Intermediate Code Representation?

Answer :

There are mainly three types of intermediate code representations.

- Syntax tree
- Postfix
- Three address code